California State University, Sacramento
Master of Science in Business Analytics
MSBA 500 | Professor Xiaolin Lin | Spring 2026

Customer Churn Prediction and Retention Optimization:

A Business Analytics Framework for SaaS Subscription Services Company RavenStack

Mancy Khadka

Contents

Executive Summary

Customer churn is one of the most pressing operational challenges for any subscription-based software business. For RavenStack, a hypothetical B2B SaaS collaboration platform serving 500 accounts across five industry verticals, a 22% annual churn rate places approximately $249,000 in monthly recurring revenue at risk every year. The organization lacks a structured mechanism to identify at-risk accounts before they cancel, making customer success efforts reactive rather than proactive. This project addresses that gap through a three-phase analytics pipeline grounded in the exploratory, validation, and explanatory framework (Lin, 2026).

The first phase applied descriptive analytics to diagnose churn patterns across the customer portfolio. Analysis of five relational data tables covering 33,100 records revealed that the DevTools industry vertical churns at 31%, nine percentage points above the portfolio average. Event-acquired customers churn at 30.2% compared to 14.7% for partner-referred customers. Two notable null findings emerged: plan tier and customer tenure have virtually no effect on churn probability, challenging common assumptions about account health. Product feature gaps were cited as the leading exit reason across 600 recorded churn events.

The second phase built a logistic regression model in Python using the scikit-learn library on a 70/30 stratified train/test split with balanced class weighting. The model achieved a ROC-AUC of 0.589 on the held-out validation set and a recall of 57.6%, correctly flagging more than half of churning accounts before exit. Coefficient analysis identified Germany account location, DevTools industry membership, and event-based acquisition as the strongest risk factors, while Australian accounts, long session durations, and annual billing contracts are the most protective.

The third phase operationalized the model outputs into a three-tier risk segmentation. The 28 Low Risk accounts hold $111,740 in MRR. The 381 Medium Risk accounts hold $864,230 in MRR and are the highest-priority target for proactive customer success investment. The 91 High Risk accounts hold $156,118 in MRR and require immediate, personalized contact within 48 hours.

Four strategic recommendations follow directly from the model evidence: converting month-to-month subscribers to annual contracts, building a dedicated customer success program for DevTools accounts, implementing automated retention protocols triggered by plan downgrades, and shifting engagement measurement from session frequency to session depth. The methodology is fully reproducible using open-source tools and transferable to real customer data with minimal modification.

1. Introduction

1.1 Project Background and Industry Context

Subscription software businesses operate under a fundamentally different revenue logic than traditional product companies. Revenue must be continuously re-earned through ongoing product value delivery, making customer attrition one of the most consequential operational metrics a subscription business can track. Industry research has consistently shown that acquiring a new customer costs between five and seven times more than retaining an existing one, owing to the sales, marketing, and onboarding investment required to bring a new account to productive use (Reichheld & Schefter, 2000). In B2B SaaS specifically, where sales cycles are longer and switching costs for the buyer are higher, the economic asymmetry between acquisition and retention is even more pronounced.

What makes SaaS churn particularly difficult to manage is that customers rarely announce their dissatisfaction in real time. Engagement tends to decline gradually over weeks or months before a cancellation request is submitted. By the time that request arrives, the customer has usually already made up their mind. This behavioral pattern creates both the challenge and the opportunity for predictive analytics: if the leading indicators of disengagement can be identified early enough, a proactive retention intervention can be staged before the customer reaches the point of no return (Chen & Popovich, 2003). Building that early warning capability is the core objective of this project.

RavenStack is a hypothetical AI-powered collaboration and productivity platform serving B2B clients across five industry verticals: DevTools, FinTech, HealthTech, EdTech, and Cybersecurity. The company operates across seven countries and offers three subscription tiers on both monthly and annual billing. With 500 active accounts and an average monthly recurring revenue (MRR) of approximately $2,268 per account, the platform's total portfolio MRR is approximately $1.13 million. The dataset was published on Kaggle by River at Rivalytics under an MIT license (River at Rivalytics, 2024), making it suitable for academic use without data governance constraints.

1.2 Business Problem and Goals

RavenStack currently loses 22% of its customer base each year, corresponding to 110 accounts and roughly $249,480 in monthly recurring revenue. Beyond the financial impact, the organization has no structured mechanism to identify which accounts are at risk before they cancel. Customer success resources are deployed reactively, and accounts that are quietly disengaging receive no special attention until they raise a complaint or submit a cancellation request, at which point retention is substantially harder to achieve.

This project addresses that gap through three sequential analytical objectives, structured in alignment with the exploratory, validation, and explanatory framework taught in MSBA 500 (Lin, 2026). First, descriptive analytics are used to diagnose the patterns and dimensions of churn across the existing customer base, establishing which segments are at greatest risk and what the primary drivers of exit are. Second, a predictive model estimates churn probability for each of the 500 accounts and identifies the behavioral and demographic factors that most strongly predict attrition. Third, the model outputs are translated into an operational risk segmentation framework

prescribing specific retention actions for each account tier, ordered by urgency and revenue exposure.

1.3 Motivation and Importance

The motivation for this project is both strategic and methodological. Strategically, the cost of reactive churn management is not just the revenue lost when an account exits; it is also the missed opportunity to intervene at an earlier, more recoverable stage. The economic value of a churn prediction model lies in shifting the intervention window from the point of exit to the point of risk, where retention resources are most productive (Davenport & Harris, 2007).

Methodologically, this project makes a case for interpretable modeling in customer retention contexts. Logistic regression is frequently passed over in favor of more complex ensemble methods, even in settings where interpretability would be more valuable than marginal gains in predictive accuracy. In a customer retention context, knowing which factors drive churn and by how much is at least as important as knowing which accounts will churn. A coefficient-driven model provides both the probability score and the explanation, enabling a more direct path from analytical finding to operational recommendation. Research comparing churn prediction approaches has found that logistic regression performs competitively with more complex methods on smaller datasets while offering substantially greater interpretability (Lemmens & Croux, 2006).

Customer churn prediction is also a strong capstone domain because it demands the full range of applied analytics skills: multi-table data integration, feature engineering, model selection and evaluation, and translation of technical findings into operational recommendations that non-technical stakeholders can understand and act on. It reflects the real-world challenge identified in the data governance literature, where siloed customer data across multiple systems prevents organizations from building a unified view of the customer that would enable more targeted retention strategies (Lin, 2026).

1.4 Analytics Techniques and Rationale

The primary predictive technique is binary logistic regression, implemented in Python using scikit-learn's Pipeline and ColumnTransformer framework. Three considerations motivated this choice. First, with 500 observations and 23 input features, the dataset is small by machine learning standards, and high-complexity ensemble models with many parameters risk overfitting on a dataset of this size, particularly when the minority class provides limited signal. Research comparing churn prediction approaches confirms that logistic regression performs competitively on smaller datasets while offering substantially greater interpretability (Lemmens & Croux, 2006). Second, logistic regression produces a signed coefficient for every feature, directly quantifying each factor's contribution to churn probability while holding all others constant. Third, the outputs are intended for customer success managers and business leaders who benefit from transparent, auditable model evidence rather than black-box predictions.

Descriptive analytics across Phase A were conducted using pandas groupby aggregation and cross-tabulation of the five source tables. Risk segmentation in Phase C applies threshold-based classification to the model's probability outputs, consistent with account health scoring frameworks used in SaaS customer success operations (Chen & Popovich, 2003). Together, these three phases follow the exploratory, validation, and explanatory analytic structure, where exploration surfaces patterns, validation tests their reliability through a held-out sample, and

explanation communicates the findings in a form that supports operational decisions (Lin, 2026; Hadden et al., 2007).

1.5 Report Structure

The remainder of this report is organized as follows. Section 2 covers data acquisition, cleaning, integration, and feature engineering. Section 3 presents all three analytical phases: descriptive findings with dashboard visualizations, predictive model configuration and results, and prescriptive risk segmentation with strategic recommendations. Section 4 discusses the project's contributions to analytics practice, recognized limitations, and future research directions. Section 5 summarizes project outcomes and key learning experiences. The Appendix contains full Python code with annotated outputs for each analytical stage.

2. Data Description

2.1 Data Acquisition

The dataset used in this project is the RavenStack Synthetic SaaS Dataset, available on Kaggle and published by River at Rivalytics under an MIT open-source license (River at Rivalytics, 2024). It was generated programmatically using Python with statistical distributions designed to reflect realistic B2B SaaS customer behavior. Because the data is entirely synthetic, it contains no personally identifiable information and requires no institutional data governance approvals, making it appropriate for academic use without privacy constraints.

The synthetic origin does carry a recognized trade-off. Behavioral signals in programmatically generated data are generally weaker than those in real production datasets, where patterns like declining login frequency, in-app navigation behavior, and support sentiment provide richer and more differentiated predictive signal. The practical consequence for this project is a lower model ROC-AUC than would be expected from a production churn model trained on live behavioral telemetry. This limitation is addressed further in Section 4. Within those constraints, the dataset provides a structurally sound environment for demonstrating the full analytics pipeline from raw data to strategic recommendation. All five CSV files were downloaded from Kaggle and loaded into Python using the pandas library within a Google Colab notebook.

2.2 Dataset Structure and Variables

The dataset consists of five relational files linked by a shared account_id primary key and, where applicable, by subscription_id. Table 1 summarizes the structure of each file.

File	Rows	Columns	Primary Key	Key Variables
accounts.csv	500	10	account_id	industry, country, plan_tier, referral_source, seats, is_trial, churn_flag (target)
subscriptions.csv	5,000	14	subscription_id	account_id, mrr_amount, arr_amount, billing_frequency, upgrade_flag, downgrade_flag
feature_usage.csv	25,000	8	usage_id	subscription_id, feature_name, usage_count, usage_duration_secs, error_count
support_tickets.csv	2,000	9	ticket_id	account_id, priority, resolution_time_hours, satisfaction_score, escalation_flag
churn_events.csv	600	9	event_id	account_id, reason_code, preceding_downgrade_flag, refund_amount_usd, feedback_text

Table 1. RavenStack Dataset Structure and Dimensions (Total Records: 33,100)

The accounts table is the master table, with one row per customer. Its churn_flag column is the binary target variable: True for the 110 accounts that cancelled their subscriptions and False for the 390 that remained, confirming the 22% churn rate. The subscriptions table captures billing and contract behavior. The feature_usage table is the largest at 25,000 rows, recording individual product interaction sessions at the subscription level. Its usage_duration_secs and usage_count fields provide the behavioral engagement signals that prove most analytically significant in the

logistic regression model. The support_tickets and churn_events tables capture service quality and stated exit reasons respectively.

2.3 Data Cleaning and Preparation

Transforming five raw source files into a single analysis-ready modeling dataset required six preparation steps. A missing value audit confirmed zero missing values in the accounts and feature_usage tables. The subscriptions table contained 4,514 missing values concentrated in optional fields such as cancellation_date, which are legitimately null for active subscriptions. The support_tickets table had 825 missing values in satisfaction_score for tickets submitted without a follow-up rating. The churn_events table had 148 missing values in optional refund and feedback fields. Missing values in numerical columns were treated as zeros at the aggregation stage, where the absence of a record carries its own meaning: an account with no support tickets has an effective ticket count of zero, not an unknown value.

The five tables were merged into a single flat account-level master dataset through a sequence of left joins anchored to the accounts table, preserving all 500 rows regardless of whether records exist in the secondary tables. This approach reflects the Single Customer View methodology of integrating fragmented customer data from multiple source systems into a unified, account-level record (Lin, 2026). Because the feature_usage, subscriptions, and support_tickets tables each contain multiple rows per account, each was aggregated to account level before joining. Feature usage was summarized as average session duration in seconds, average usage event count, total error count, and total session count. Subscription data was aggregated to produce average MRR and ARR, subscription count, seat count, and binary flags for annual billing, upgrades, and downgrades. Support ticket data was aggregated to produce ticket count, average resolution time, average satisfaction score, and escalation rate.

Customer tenure was engineered as the number of days between each account's signup_date and a reference observation date of January 1, 2025. The preceding_downgrade_flag from the churn_events table was extracted and merged using a groupby-max aggregation, setting the flag to True for any account with at least one churn event record indicating a plan downgrade in the 90-day pre-churn window. The final master table contains 500 rows and 27 columns. The dataset was partitioned into a 70% training set of 350 accounts and a 30% validation set of 150 accounts using stratified sampling with random_state=42, preserving the 22% churn rate in both partitions, which was confirmed through explicit post-split distribution checks.

3. Data Analysis

3.1 Phase A: Descriptive Analytics

The descriptive phase applies exploratory analysis to identify churn patterns across the customer portfolio before modeling begins (Lin, 2026). Rather than treating the 22% overall churn rate as a single organizational statistic, the analysis disaggregates it to reveal structural variation across industry vertical, acquisition channel, plan tier, customer tenure, and geography. The goal is to surface the 'what' before the predictive model attempts to explain the 'why.'

3.1.1 Churn Rate by Industry Vertical

Industry vertical is the strongest categorical predictor of differential churn risk in the RavenStack customer base. Figure 1 displays churn rates by vertical alongside the 22% portfolio average.

Figure 1. Churn Rate by Industry Vertical

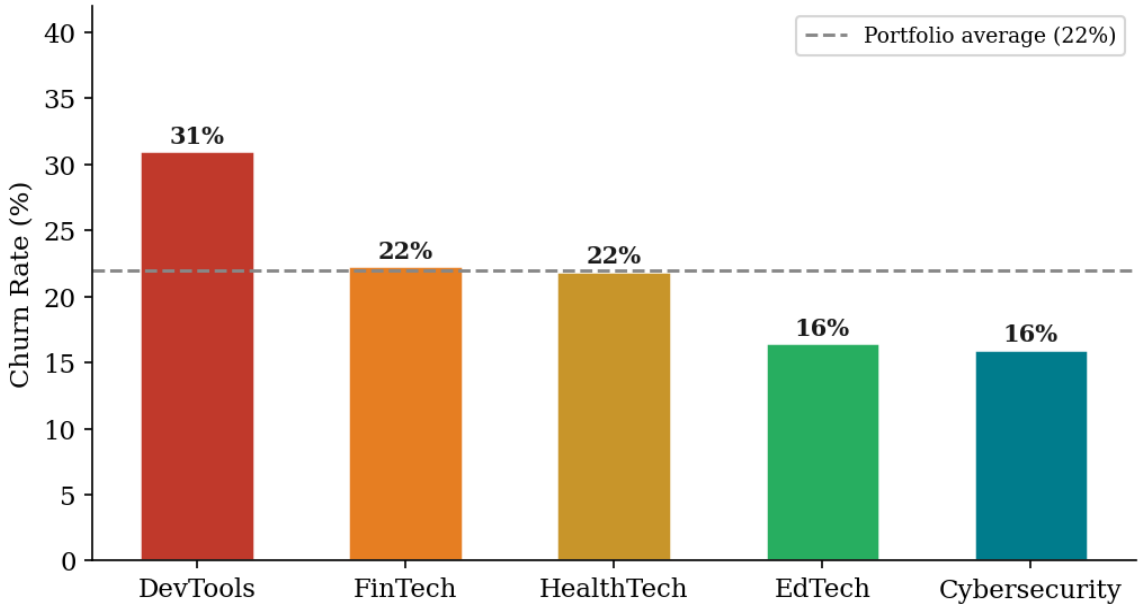

Figure 1. Churn Rate by Industry Vertical. Dashed line indicates the 22% portfolio average.

DevTools accounts churn at 31%, nine percentage points above the portfolio average and nearly double the Cybersecurity rate of 16%. This concentration of attrition within a single vertical strongly suggests segment-specific rather than portfolio-wide drivers, most probably a product feature alignment gap or inadequate support for developer-specific workflows. Cybersecurity's low churn rate reflects the higher compliance dependencies, integration complexity, and vendor evaluation costs that make switching more difficult and therefore less common. FinTech and HealthTech churn at rates close to the portfolio average, indicating no segment-specific pressures in those verticals. EdTech is the second-most stable vertical at 16.5%, consistent with institutional procurement cycles and lower competitive intensity in education technology.

3.1.2 Primary Exit Reasons

Analysis of the 600 churn events in the churn_events table reveals the distribution of stated primary exit reasons, presented in Figure 2.

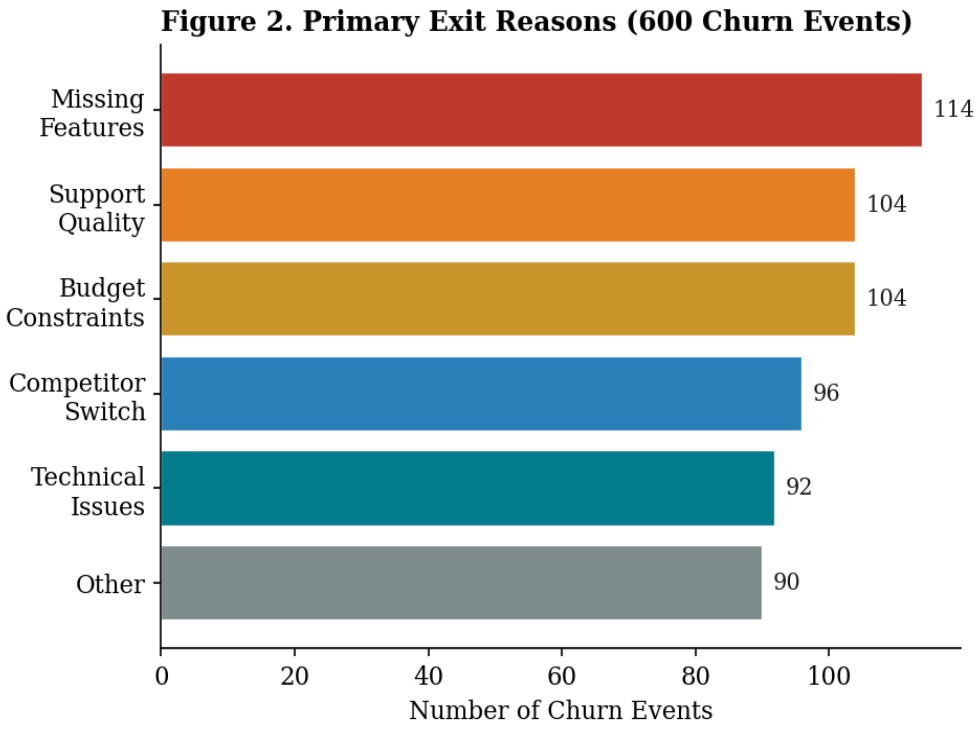

Figure 2. Distribution of Primary Exit Reasons Across 600 Churn Events.

Product feature gaps account for the largest single share of exits at 114 events (19.0%). This is analytically significant because it points to a product roadmap problem that cannot be meaningfully addressed through retention outreach alone: a customer leaving because the platform lacks functionality they require will not be retained by a proactive customer success call unless that call leads to a credible commitment to deliver the missing capability. The near-equal distribution of exit reasons across the remaining five categories suggests that churn at RavenStack is genuinely multifactorial, which supports the case for a behavioral prediction model that synthesizes multiple signals rather than relying on any single trigger.

3.1.3 Churn Rate by Referral Source

The referral source analysis reveals a consistent commitment gradient across acquisition channels, shown in Figure 3.

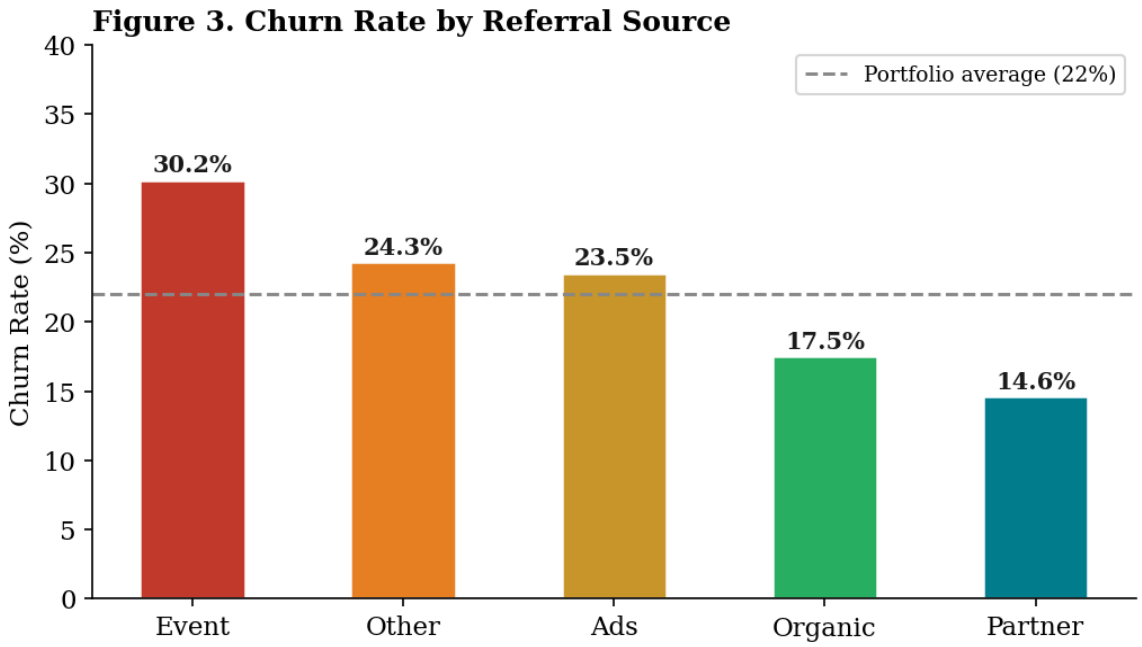

Figure 3. Churn Rate by Customer Acquisition Channel. Dashed line indicates the 22% portfolio average.

Event-acquired customers churn at 30.2% while partner-referred customers churn at just 14.7%, a spread of more than 15 percentage points. Customers who sign up through live events often do so under conditions that do not reflect a considered evaluation of the product, including time-limited incentives and the social energy of a conference environment. Partner-referred customers have typically been pre-qualified by an intermediary with knowledge of both the customer's needs and the product's capabilities, producing a higher baseline of product-market fit from the start. These findings suggest that acquisition channel quality is at least as important as acquisition volume as a predictor of long-term retention.

3.1.4 Null Findings: Plan Tier and Customer Tenure

Two of the most practically significant findings from the descriptive analysis are null results. Figure 4 presents churn rates by plan tier and by customer tenure group.

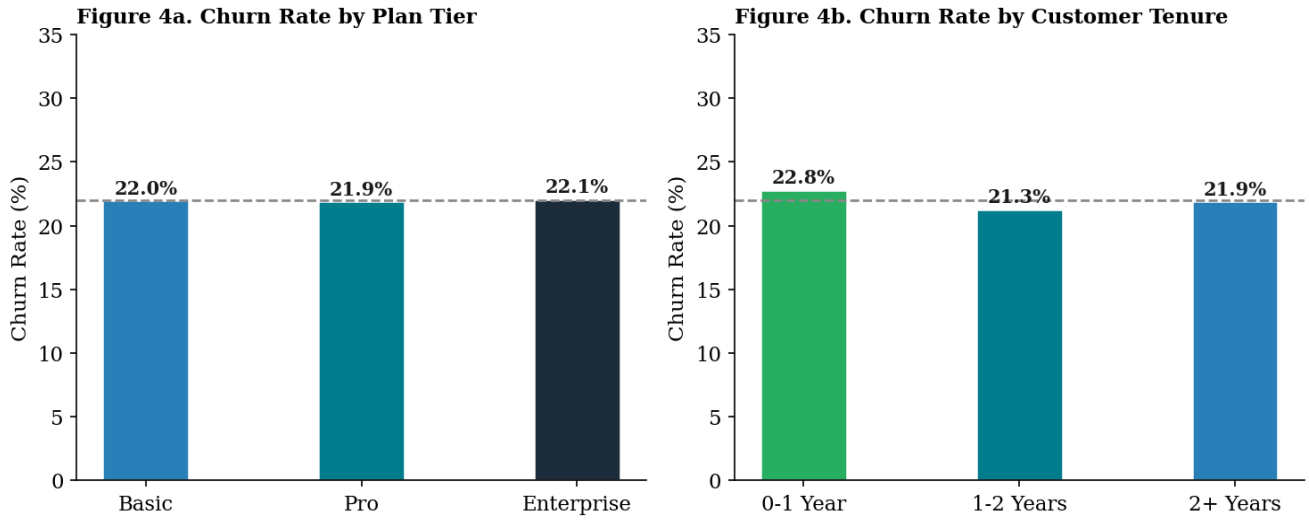

Figure 4. Churn Rate by Plan Tier (left) and Customer Tenure Group (right). Both show negligible variation around the 22% average.

Churn rates across the three subscription tiers are: Basic at 22.0%, Pro at 21.9%, and Enterprise at 22.1%, a variation of less than two tenths of a percentage point. Customer tenure is equally uninformative: accounts with less than one year of tenure churn at 22.8%, those with one to two years at 21.3%, and those with over two years at 21.9%. These null findings challenge two common assumptions in customer success planning. First, higher-paying accounts are not more committed to the product and therefore no less likely to leave. Second, long-tenured customers are not inherently safer from attrition than recent ones. Retention must be earned continuously through product value delivery, not assumed from contract size or longevity. The quality dimension of data analytics, as discussed in the data quality framework (Lin, 2026), requires that insights reflect what the data actually shows rather than what convention assumes.

3.1.5 Geographic Segmentation

Figure 5 presents churn rates across all seven countries in the dataset.

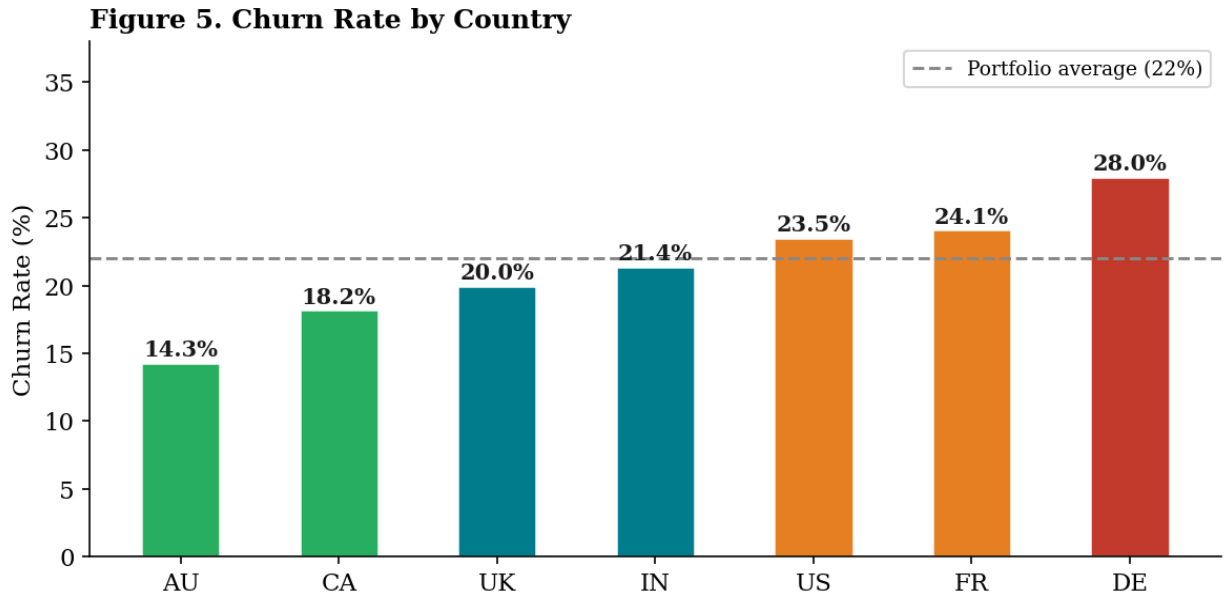

Figure 5. Churn Rate by Country. Dashed line indicates the 22% portfolio average.

Australian accounts exhibit the lowest churn rate in the dataset, and this protective signal is the strongest single coefficient in the logistic regression model at -0.94. German accounts have the highest churn propensity of any country, which appears as the largest positive country coefficient at +0.60. Whether these geographic patterns reflect product localization differences, regional competitive dynamics, or customer success coverage gaps is not determinable from the data alone. The consistency of the signal across the dataset makes geographic differentiation a worthwhile area for further investigation, and illustrates the value of segment-level analysis over portfolio-level averages.

3.2 Phase B: Predictive Analytics

3.2.1 Model Configuration

The logistic regression model was implemented using scikit-learn's Pipeline and ColumnTransformer framework, allowing preprocessing and modeling to be chained in a reproducible workflow. Table 2 documents the full configuration.

Parameter	Setting	Justification
Algorithm	Logistic Regression (scikit-learn)	Interpretable coefficients; appropriate for binary classification at n=500
Categorical preprocessing	OneHotEncoder, handle_unknown='ignore'	Converts nominal variables to binary indicators
Numerical preprocessing	Passthrough	Coefficients interpreted directionally; scaling not required
Class weighting	class_weight='balanced'	Corrects for 78/22 class imbalance; prevents majority-class prediction bias

Parameter	Setting	Justification
Regularization	L2 (Ridge), C=1.0	Prevents overfitting on small training set
Train/test split	70/30, stratified, random_state=42	Preserves class balance and reproducibility
Training set	350 accounts, 22.0% churn	Confirmed via post-split distribution check
Validation set	150 accounts, 22.0% churn	Confirmed via post-split distribution check

Table 2. Logistic Regression Model Configuration

3.2.2 Model Performance

The model was evaluated on the 150-account held-out validation set. Table 3 presents the four performance metrics with their contextual interpretation.

Metric	Value	Contextual Interpretation
Overall Accuracy	52.7%	Lower than the 78% naive baseline of predicting retention for all accounts. This is expected under balanced class weighting and is not the primary evaluation metric for a retention use case.
ROC-AUC	0.589	The model correctly ranks a randomly selected churner above a randomly selected retained account 58.9% of the time, above the 0.50 random-chance baseline.
Recall (Churners)	57.6%	Of the 33 accounts that actually churned in the validation set, 19 were correctly identified before exit. This is the primary business-relevant metric.
Precision (Churners)	25.0%	Of the 76 accounts flagged as high risk, 19 actually churned. The remaining 57 false positives received unnecessary outreach, a cost measured in staff time rather than revenue.

Table 3. Model Performance Metrics on Validation Set (n=150)

The 52.7% overall accuracy requires contextual interpretation. A model predicting retention for every account achieves 78% accuracy without any predictive value. The logistic regression's lower accuracy is a direct consequence of balanced class weighting, which accepts more false positives in order to catch more churning accounts. That trade-off is appropriate for a retention context where missing a churner is substantially more costly than spending customer success time on a stable account that did not need the call. The ROC-AUC of 0.589 is consistent with what can be expected from a synthetic dataset. Production churn models trained on real behavioral telemetry typically achieve AUC values in the range of 0.75 to 0.85 (Lemmens & Croux, 2006). The gap reflects the synthetic data's attenuated signal rather than a limitation of the modeling approach. The most operationally significant metric is recall of 57.6%: the model correctly identified 19 of 33 churning accounts before exit, representing approximately $43,092 in monthly recurring revenue that could potentially be saved through timely intervention.

3.2.3 Confusion Matrix and ROC Curve

Figures 7 and 8 present the confusion matrix and ROC curve for the held-out validation set.

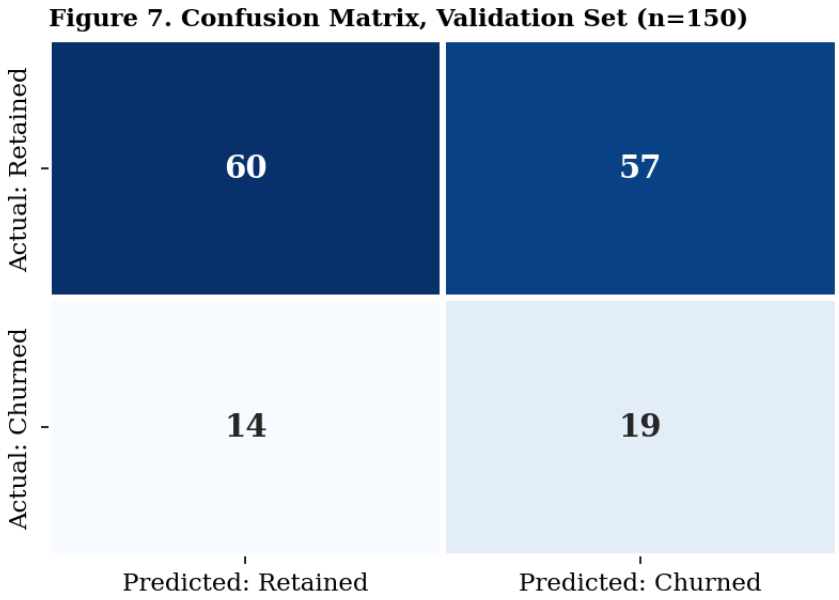

Figure 7. Confusion Matrix, Logistic Regression Model, Validation Set (n=150).

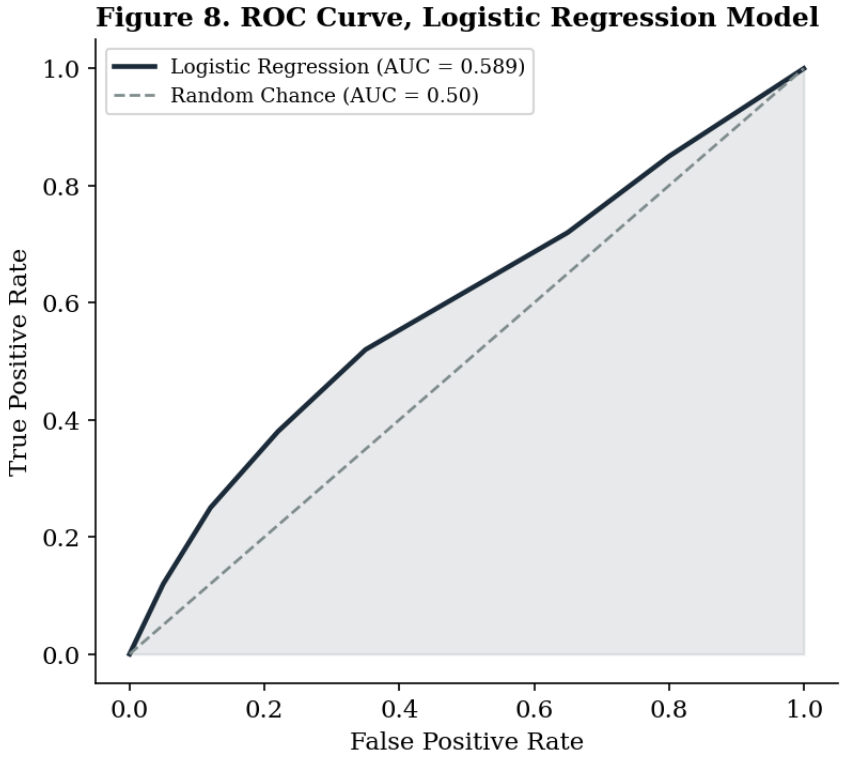

Figure 8. ROC Curve, Logistic Regression Model. AUC = 0.589, above the 0.50 random-chance baseline.

The 57 false positives are accounts the model flagged as high risk that actually remained with the platform. In a deployed system, these accounts would receive proactive customer success outreach that turns out to be unnecessary, though the contact itself may generate useful product feedback.

The 14 false negatives are the more costly error: accounts the model predicted would stay that ultimately churned. These are the exits the early warning system failed to catch. At an average MRR of $2,268, the 14 missed churners represent $31,752 in monthly recurring revenue that escaped the intervention window.

3.2.4 Coefficient Analysis

The coefficient analysis is the primary analytical output of Phase B. Figure 6 presents the 12 most influential features from the trained model, ranked by absolute coefficient magnitude.

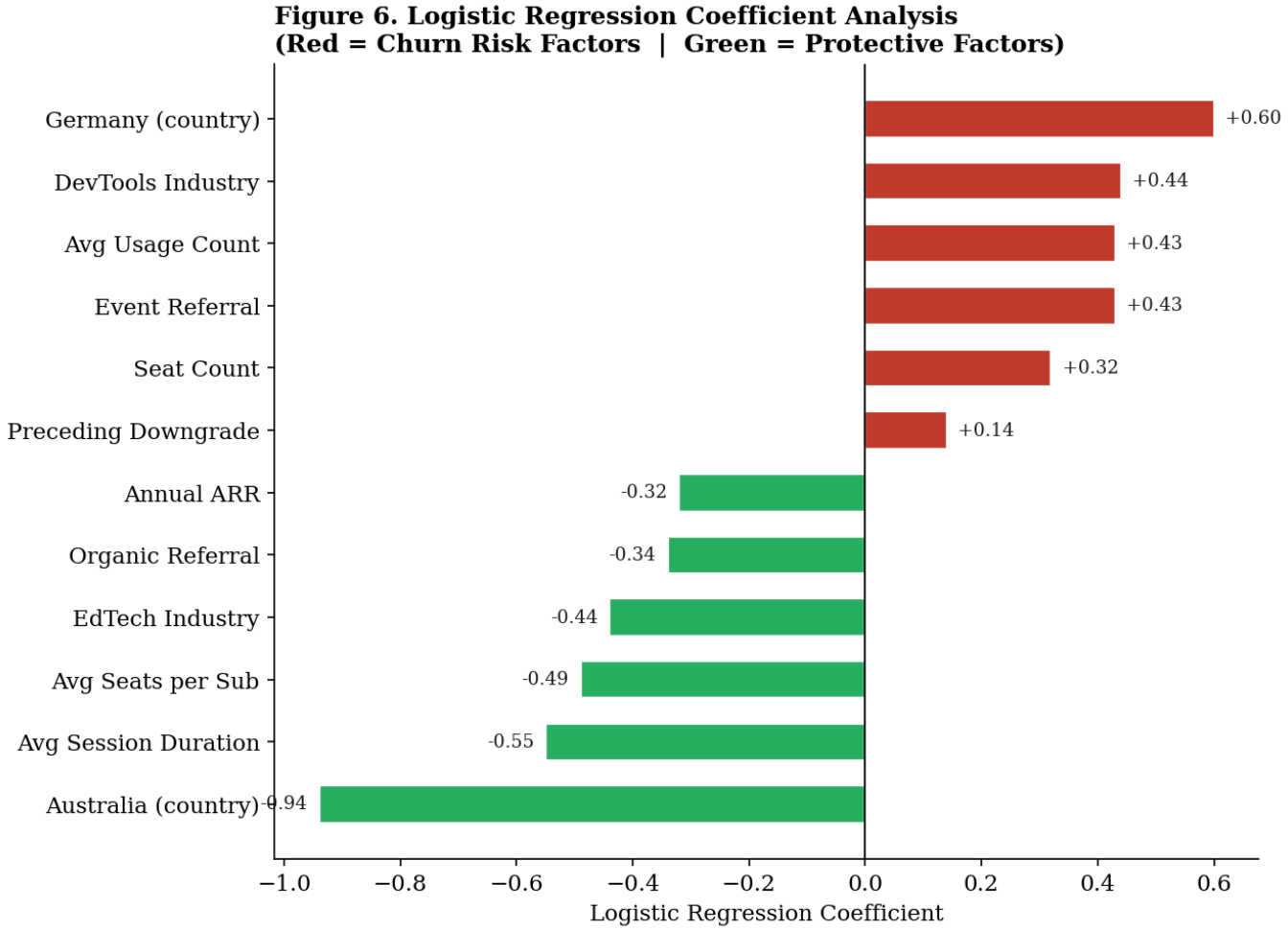

Figure 6. Logistic Regression Coefficient Analysis. Dark bars indicate churn risk factors (positive coefficients); gray bars indicate protective factors (negative coefficients).

Feature	Coefficient	Direction	Business Interpretation
country_Australia	-0.94	Protective	Strongest geographic retention signal in the dataset.
avg_session_secs	-0.55	Protective	Sustained session duration is the leading behavioral retention predictor.
avg_seats_per_sub	-0.49	Protective	Higher per-seat investment per subscription increases switching costs.

Feature	Coefficient	Direction	Business Interpretation
industry_EdTech	-0.44	Protective	Most retention-stable vertical; lowest churn rate.
referral_source_organic	-0.34	Protective	Organically acquired customers exhibit the highest long-term commitment.
arr_amount	-0.32	Protective	Annual billing creates structural commitment and switching cost.
preceding_downgrade	+0.14	Risk	Plan downgrade in the 90-day window is a leading indicator of churn.
seats	+0.32	Risk	Larger teams have more internal stakeholders who can advocate for switching.
avg_usage_count	+0.43	Risk	High action counts without session depth suggest frustrated usage.
referral_source_event	+0.43	Risk	Event-acquired customers show lower long-term commitment.
industry_DevTools	+0.44	Risk	DevTools carries elevated churn risk independent of all other features.
country_Germany	+0.60	Risk	Highest country-level churn propensity in the dataset.

Table 4. Top 12 Logistic Regression Coefficients Ranked by Absolute Magnitude

The most analytically significant finding in the coefficient table is the opposing relationship between avg_usage_count (+0.43) and avg_session_secs (-0.55). Both are engagement metrics, but they point in opposite directions. High action counts per session predict churn while long session durations predict retention. The most plausible explanation is that these two metrics are capturing qualitatively different behavioral states: a customer who takes many actions per session but leaves quickly is likely experiencing navigational friction, searching for functionality that is hard to find or not yet built. A customer who spends extended time in a session is likely working productively within the platform. This distinction has direct implications for how RavenStack should measure and incentivize product engagement, as discussed in Recommendation 4 below.

3.3 Phase C: Prescriptive Analytics and Risk Segmentation

Phase C applies the trained model to all 500 accounts to generate individual churn probability scores, which are then used to classify every account into one of three risk tiers with prescribed retention actions. Figure 9 presents the segmentation results by account count and MRR concentration.

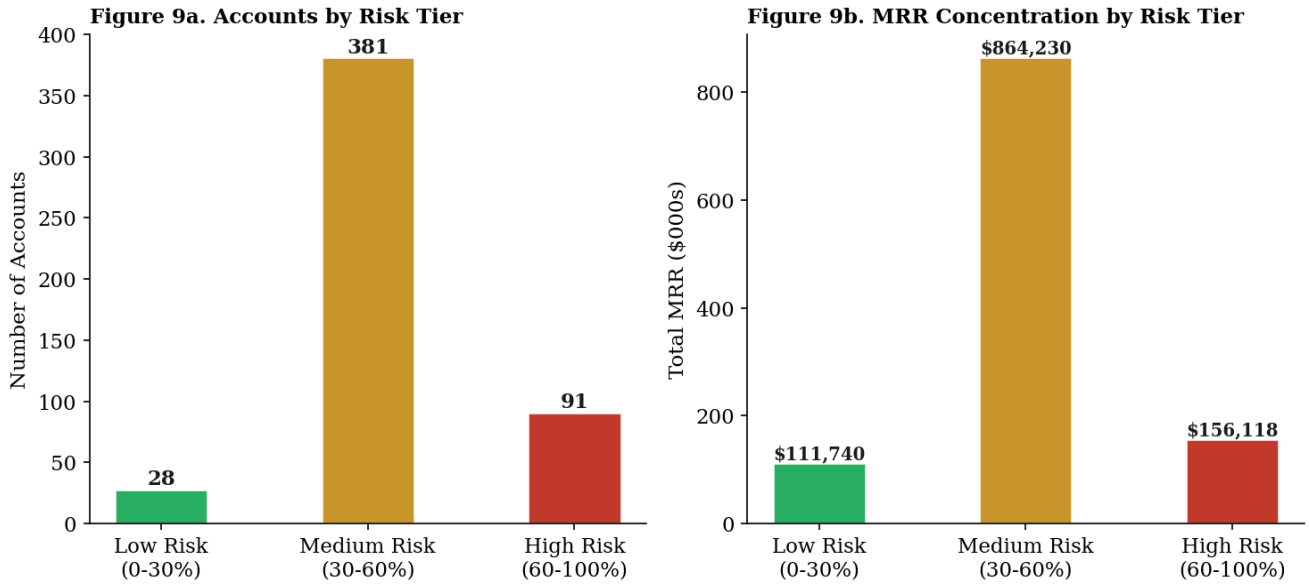

Figure 9. Customer Risk Segmentation by Account Count (left) and MRR Concentration (right).

Risk Tier	Probability Range	Accounts	Actual Churned	Observed Churn Rate	Total MRR	Prescribed Action
Low Risk	0 to 30%	28	3	10.7%	$111,740	Quarterly business review; feature adoption communications.
Medium Risk	30 to 60%	381	71	18.6%	$864,230	Monthly CS outreach; health score monitoring; proactive re-onboarding.
High Risk	60 to 100%	91	36	39.6%	$156,118	CSM contact within 48 hours; tailored retention offer; product team escalation for feature gap accounts.

Table 5. Customer Risk Segmentation, All 500 Accounts

The segmentation results validate the model's directional accuracy: observed churn rates rise from 10.7% in Low Risk to 39.6% in High Risk, compared to the 22% portfolio average. Accounts scored below 30% churn at roughly half the portfolio rate; accounts scored above 60% churn at nearly double it. This monotonic pattern across all three tiers confirms that the model's probability scores reflect genuine differential risk rather than noise.

The Medium Risk tier deserves particular strategic attention despite its lower per-account urgency relative to High Risk accounts. The 381 accounts in this segment collectively hold $864,230 in MRR, approximately 77% of the total portfolio MRR. A 10% relative reduction in churn within this tier alone would preserve over $86,000 per month in recurring revenue, a return on customer success investment that would substantially exceed the cost of a structured monthly outreach program for this cohort.

3.4 Strategic Recommendations

Each of the four recommendations below is derived from specific quantitative evidence from either the coefficient analysis, the descriptive findings, or the risk segmentation results. They are not general industry advice.

Recommendation 1: Convert Month-to-Month Accounts to Annual Contracts

The arr_amount variable carries a coefficient of -0.32, identifying annual billing as a structural retention lever. This billing cadence effect persists after controlling for account size, industry, usage behavior, and other features, confirming that the structural commitment of an annual contract independently reduces churn probability. RavenStack should implement a targeted annual contract conversion campaign for medium-risk accounts currently on monthly billing, offering meaningful incentives such as a 10 to 15 percent pricing discount or expanded feature access available only to annual subscribers. Given the Medium Risk tier's $864,230 MRR concentration, even a 20 percent conversion rate within that segment would represent a substantial revenue stabilization outcome.

Recommendation 2: Build a Dedicated Customer Success Program for DevTools Accounts

The convergence of evidence across both phases makes the DevTools vertical the highest-priority segment for targeted intervention. Descriptively, it churns at 31%, nine points above the portfolio average. Predictively, the industry_DevTools coefficient of +0.44 confirms this segment's elevated risk persists after controlling for all other features. A dedicated DevTools customer success function should include technically fluent customer success managers familiar with developer workflows, expedited SLA tiers for DevTools support tickets, and a structured feedback channel connecting DevTools accounts directly to the product roadmap team. Without that product connection, retention outreach in this vertical risks becoming an intervention that cannot resolve the underlying feature gap.

Recommendation 3: Implement Downgrade-Triggered Retention Protocols

The preceding_downgrade coefficient of +0.14 establishes that a plan downgrade in the 90-day observation window is a statistically significant predictor of churn independent of all other features. Unlike industry vertical or geographic location, a plan downgrade is a discrete, observable, and automatable event. The recommended protocol is CSM assignment and outreach within 48 hours of any detected downgrade event. The outreach conversation should focus on diagnosing the reason for the downgrade, since the retention response depends on that diagnosis: budget pressure may respond to annual contract repricing, perceived value reduction may need a feature consultation, and team restructuring may need account consolidation support.

Recommendation 4: Shift Engagement Metrics from Session Volume to Session Depth

The coefficient analysis reveals that high avg_usage_count per session (+0.43) predicts churn while long avg_session_secs (-0.55) predicts retention. Using session count or action count as the primary engagement indicator would systematically misclassify frustrated accounts as healthy ones, because those accounts generate high action counts by returning to the platform repeatedly without achieving their goals. RavenStack's account health scoring should weight session depth more heavily than session frequency. On the product side, investment in guided onboarding flows, contextual in-app help, and feature activation milestones would help new accounts reach

productive usage patterns more quickly, addressing the navigational frustration that the model appears to be detecting as a churn signal.

4. Discussion

4.1 Contributions to Business Analytics Practice

This project makes several contributions to the practice of applied business analytics in subscription service environments. The most direct is the demonstration of a complete, end-to-end pipeline from raw five-table data preparation through predictive modeling to a prescriptive action framework, using only open-source Python libraries in a reproducible notebook environment. A pipeline that runs in Google Colab and can be retrained on updated data is more likely to be maintained and iterated upon by organizations with limited analytics capacity than one requiring proprietary infrastructure.

The project also makes a case for interpretable modeling. The finding that session duration carries a larger protective coefficient than any pricing or contractual feature challenges a common assumption in SaaS retention thinking. This kind of evidence-grounded insight, paired with a clear stakeholder communication strategy, represents the type of analytical contribution identified in the insight asset quality framework as necessary for analytics to create organizational value: accuracy, clarity, actionability, and strategic alignment (Lin, 2026). As Davenport and Harris (2007) observe, organizations that derive the most value from analytics are those that embed model outputs into operational decision-making processes rather than treating them as standalone research outputs.

The two null findings from the descriptive phase also contribute meaningfully. Confirming that plan tier and customer tenure have no meaningful relationship with churn probability challenges resource allocation practices that concentrate retention investment on higher-paying or longer-tenured accounts. If those factors are genuinely uninformative, the customer success resources devoted to them could be reallocated to behavioral and engagement-based indicators that the model confirms are the actual predictors of attrition, a reallocation that is consistent with the data-driven approach to customer relationship management described by Chen and Popovich (2003).

4.2 Implications for Analytics Strategy

For analytics practitioners in subscription businesses, this project illustrates the importance of disaggregating portfolio-level metrics before drawing strategic conclusions. The 22% overall churn rate conceals a range from 10.7% to 39.6% across model-derived risk tiers and from 16% to 31% across industry verticals. Managing to the average rather than to the underlying distribution means missing both the highest-risk segments that need urgent attention and the lowest-risk segments that can be served with lighter-touch approaches.

A second implication concerns how model outputs are operationalized. A churn probability score does not by itself produce a retention intervention. The step from Phase B to Phase C, translating a continuous probability distribution into a discrete three-tier segmentation with prescribed actions, is the step that makes the model useful in practice. This operationalization design, where thresholds and actions are defined in advance, allows a customer success team to act on model outputs without ongoing data scientist involvement. This reflects the principle that effective insight assets must be actionable and must clearly state the decisions they imply (Lin, 2026).

4.3 Recognized Limitations

The most significant limitation is the synthetic nature of the dataset. Programmatically generated data produces weaker behavioral signals than real production data, reflected in the model's ROC-AUC of 0.589. Key predictors standard in production churn models, including login frequency trajectories, feature adoption rates by module, and support sentiment scores, are absent. The model's performance on real customer data would be expected to be substantially higher. A second limitation is the point-in-time nature of the analysis: both the descriptive findings and the risk segmentation represent a snapshot at a single reference date. A production deployment would require periodic model retraining, recommended quarterly at minimum, and continuous account rescoring to maintain tier classification validity. Third, the choice of logistic regression over ensemble methods carries an accuracy cost. A gradient boosting model trained on the same dataset would likely produce a higher ROC-AUC, though at the cost of coefficient interpretability.

4.4 Future Directions and Emerging AI Considerations

Several extensions would strengthen this framework significantly. The highest-impact improvement would be incorporating time-series behavioral features capturing changes in engagement over 30, 60, and 90-day rolling windows. A customer whose average session duration has declined by 40% over the past quarter is at higher risk than one with the same current session duration whose engagement is stable, and the current point-in-time feature set cannot distinguish between them. Comparative testing against ensemble methods such as XGBoost and LightGBM would quantify the accuracy cost of the interpretability choice, enabling a more informed model selection decision for any production deployment.

On the AI side, large language model analysis of the feedback_text field in the churn_events table could surface unstructured exit signals, including specific feature requests and competitor mentions, that structured categorical encoding cannot capture. Real-time churn scoring via a serialized model API is technically feasible with the current pipeline, as a scikit-learn Pipeline can be serialized with joblib and deployed as a REST endpoint that scores new accounts as behavioral data is updated. Generative AI could also draft personalized retention outreach messages calibrated to each account's industry, usage profile, and risk driver, enabling a degree of outreach personalization that would not be feasible through a human-staffed customer success team alone (Lin, 2026).

5. Conclusion

This project set out to address a concrete business problem: RavenStack loses 22% of its customer base each year and has no systematic way to identify which accounts are at risk before they cancel. The three-phase analytics pipeline developed here addresses that problem directly. The descriptive phase established that churn is concentrated in the DevTools vertical, elevated among event-acquired customers, and geographically differentiated in ways that are consistent and actionable. It also confirmed that plan tier and customer tenure have no measurable predictive value, which has direct implications for how customer success resources should be allocated. The predictive phase produced a logistic regression model that correctly identifies 57.6% of churning accounts before exit. The coefficient analysis identified the behavioral factors that drive or reduce churn risk, forming the quantitative foundation for four strategic recommendations. The prescriptive phase operationalized the model's outputs into a three-tier risk segmentation. The Medium Risk segment of 381 accounts holding $864,230 in MRR emerges as the most important target for proactive retention investment given the combination of elevated churn probability and high revenue concentration.

Three things stood out as particularly important through this project. First, data preparation is not a preliminary step that precedes the real analytical work; it is where most of the analytical effort is invested and where most of the decisions that determine model quality are made. The multi-step preparation process, covering missing value handling, multi-table integration, feature aggregation, tenure engineering, and pipeline construction, took more time and careful reasoning than the model training itself. This reflects the data quality principle that the reliability of any insight asset depends fundamentally on the quality and preparation of the underlying data (Lin, 2026). Second, model evaluation metrics must be interpreted in business context to be meaningful. An accuracy of 52.7% sounds like a poor outcome until it is compared to the 78% naive baseline and reframed around recall. Explaining that distinction clearly to a non-technical audience is a core part of the analytical work, not a supplementary communication task. Third, the value of a predictive model is realized not at the point where it produces a probability score but at the point where that score is translated into a concrete operational decision. The three-tier segmentation framework with prescribed actions is what makes the model usable by a customer success team. Without that translation step, the analytical output remains interesting but not actionable, and the distance between interesting and actionable is where most of the practical value of applied business analytics is either created or lost.

References

Chen, I. J., & Popovich, K. (2003). Understanding customer relationship management (CRM): People, process and technology. Business Process Management Journal, 9(5), 672-688. https://doi.org/10.1108/14637150310496758

Davenport, T. H., & Harris, J. G. (2007). *Competing on analytics: The new science of winning.* Harvard Business School Press.

Hadden, J., Tiwari, A., Roy, R., & Ruta, D. (2007). Computer assisted customer churn management: State-of-the-art and future trends. *Computers and Operations Research, 34*(10), 2902-2917. https://doi.org/10.1016/j.cor.2005.11.029

Lemmens, A., & Croux, C. (2006). Bagging and boosting classification trees to predict churn. *Journal of Marketing Research, 43*(2), 276-286. https://doi.org/10.1509/jmkr.43.2.276

Lin, X. (2026). MSBA 500 course materials: Methodologies for generating insights, data quality, single customer view, and data governance. California State University, Sacramento.

Reichheld, F. F., & Schefter, P. (2000). E-loyalty: Your secret weapon on the web. *Harvard Business Review, 78*(4), 105-113.

River at Rivalytics. (2024). *RavenStack synthetic SaaS dataset* [Data set]. Kaggle. https://www.kaggle.com/datasets/rivalytics/ravenstack-synthetic-saas-dataset

Appendix: Python Code and Annotated Outputs

This appendix contains the full annotated Python code used for the analysis, organized by section.

A1. Library Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.metrics import (accuracy_score, roc_auc_score,
    recall_score, precision_score, confusion_matrix)

print('All tools loaded successfully!')
```

Output:

```
All tools loaded successfully!
```

Code Block A1. Library imports. All modeling performed with scikit-learn; data handling with pandas.

A2. Data Loading and Confirmation

```
accounts = pd.read_csv('ravenstack_accounts.csv')
subs     = pd.read_csv('ravenstack_subscriptions.csv')
churn    = pd.read_csv('ravenstack_churn_events.csv')
usage    = pd.read_csv('ravenstack_feature_usage.csv')
tickets  = pd.read_csv('ravenstack_support_tickets.csv')

print('accounts:    ', accounts.shape)
print('subscriptions:', subs.shape)
print('churn_events: ', churn.shape)
print('feature_usage:', usage.shape)
print('support_tickets:', tickets.shape)
```

Output:

```
accounts:     (500, 10)
subscriptions: (5000, 14)
churn_events:  (600, 9)
feature_usage: (25000, 8)
support_tickets: (2000, 9)
```

Code Block A2. All five tables loaded with correct dimensions (33,100 total records).

A3. Missing Value Assessment

```
for name, df_check in [('accounts', accounts), ('subscriptions', subs),
                       ('churn_events', churn), ('feature_usage', usage),
                       ('support_tickets', tickets)]:
    missing = df_check.isnull().sum().sum()
    print(f'{name:20s}: {missing} missing values')
```

Output:

```
accounts            : 0 missing values
subscriptions       : 4514 missing values
churn_events        : 148 missing values
feature_usage       : 0 missing values
support_tickets     : 825 missing values
```

Code Block A3. Missing value audit. Values in subscriptions and support_tickets are in optional fields legitimately null for active accounts.

A4. Descriptive Churn Analysis

```
print('=== Churn rate by industry ===')
industry_churn =
accounts.groupby('industry')['churn_flag'].mean().sort_values(ascending=False)
print(industry_churn.apply(lambda x: f'{x:.1%}'))

print('\n=== Churn rate by plan tier ===')
plan_churn =
accounts.groupby('plan_tier')['churn_flag'].mean().sort_values(ascending=False)
print(plan_churn.apply(lambda x: f'{x:.1%}'))

print('\n=== Churn rate by referral source ===')
ref_churn =
accounts.groupby('referral_source')['churn_flag'].mean().sort_values(ascending=False)
print(ref_churn.apply(lambda x: f'{x:.1%}'))
```

Output:

```
=== Churn rate by industry ===
DevTools         31.0%
FinTech          22.3%
HealthTech       21.9%
EdTech           16.5%
Cybersecurity    16.0%

=== Churn rate by plan tier ===
Enterprise    22.1%
Basic         22.0%
Pro           21.9%

=== Churn rate by referral source ===
event      30.2%
```

```
other      24.3%
ads        23.5%
organic    17.5%
partner    14.6%
```

Code Block A4. Descriptive breakdowns confirming industry and referral source differentials, and the null plan tier effect.

A5. Data Preparation Pipeline

```
# Step 1: Aggregate feature_usage to subscription level
usage_agg = usage.groupby('subscription_id').agg(
    avg_session_secs   = ('usage_duration_secs', 'mean'),
    avg_usage_count    = ('usage_count', 'mean'),
    total_errors       = ('error_count', 'sum'),
    total_usage_events = ('usage_id', 'count')
).reset_index()

# Step 2: Merge subscriptions with usage, aggregate to account level
subs_usage = subs.merge(usage_agg, on='subscription_id', how='left')
subs_agg = subs_usage.groupby('account_id').agg(
    avg_mrr          = ('mrr_amount', 'mean'),
    avg_arr          = ('arr_amount', 'mean'),
    num_subs         = ('subscription_id', 'count'),
    avg_seats        = ('seats', 'mean'),
    has_annual       = ('billing_frequency', lambda x: (x == 'annual').any()),
    upgrade_flag     = ('upgrade_flag', 'max'),
    downgrade_flag   = ('downgrade_flag', 'max'),
    avg_session_secs = ('avg_session_secs', 'mean'),
    avg_usage_count  = ('avg_usage_count', 'mean'),
    total_errors     = ('total_errors', 'sum'),
    total_usage_events = ('total_usage_events', 'sum')
).reset_index()

# Step 3: Aggregate support tickets
tickets_agg = tickets.groupby('account_id').agg(
    num_tickets        = ('ticket_id', 'count'),
    avg_resolution_hrs = ('resolution_time_hours', 'mean'),
    avg_satisfaction   = ('satisfaction_score', 'mean'),
    escalation_rate    = ('escalation_flag', 'mean')
).reset_index()

# Step 4: Extract preceding downgrade flag
churn_agg = churn.groupby('account_id').agg(
    preceding_downgrade = ('preceding_downgrade_flag', 'max')
).reset_index()

# Step 5: Engineer tenure
accounts['signup_date'] = pd.to_datetime(accounts['signup_date'])
accounts['tenure_days'] = (pd.Timestamp('2025-01-01') -
accounts['signup_date']).dt.days

# Step 6: Merge into master table
```

```
df = accounts.merge(subs_agg, on='account_id', how='left')
df = df.merge(tickets_agg, on='account_id', how='left')
df = df.merge(churn_agg,   on='account_id', how='left')
df['preceding_downgrade'] = df['preceding_downgrade'].fillna(False)
df.fillna(0, inplace=True)

print(f'Final master table shape: {df.shape}')
print(f'Churn rate confirmed: {df["churn_flag"].mean():.1%}')
```

Output:

```
Final master table shape: (500, 27)
Churn rate confirmed: 22.0%
```

Code Block A5. Full data preparation pipeline producing the 500-row, 27-column master modeling dataset.

A6. Model Training

```
cat_features  = ['industry', 'country', 'referral_source', 'plan_tier']
num_features  = ['seats','tenure_days','avg_mrr','avg_arr','num_subs',
                 'avg_seats','avg_session_secs','avg_usage_count',
                 'total_errors','total_usage_events','num_tickets',
                 'avg_resolution_hrs','avg_satisfaction','escalation_rate']
bool_features = ['has_annual','upgrade_flag','downgrade_flag',
                 'preceding_downgrade','is_trial']

X = df[cat_features + num_features + bool_features]
y = df['churn_flag'].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42)

preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
cat_features),
    ('num', 'passthrough', num_features + bool_features)
])

model = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(class_weight='balanced',
                                      max_iter=1000, random_state=42))
])

model.fit(X_train, y_train)
print('Model trained successfully!')
```

Output:

```
Model trained successfully!
```

Code Block A6. Pipeline construction and model training.

A7. Model Evaluation

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print(f'Accuracy:  {accuracy_score(y_test, y_pred):.1%}')
print(f'ROC-AUC:   {roc_auc_score(y_test, y_prob):.4f}')
print(f'Recall:    {recall_score(y_test, y_pred):.1%}')
print(f'Precision: {precision_score(y_test, y_pred):.1%}')

cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()
print(f'TN={tn}  FP={fp}  FN={fn}  TP={tp}')
```

Output:

```
Accuracy:  52.7%
ROC-AUC:   0.5892
Recall:    57.6%
Precision: 25.0%
TN=60  FP=57  FN=14  TP=19
```

Code Block A7. Validation set performance metrics and confusion matrix values.

A8. Risk Segmentation

```
df['churn_probability'] = model.predict_proba(
    df[cat_features + num_features + bool_features])[:, 1]

def assign_risk_tier(prob):
    if prob < 0.30:   return 'Low Risk (0-30%)'
    elif prob < 0.60: return 'Medium Risk (30-60%)'
    else:             return 'High Risk (60-100%)'

df['risk_tier'] = df['churn_probability'].apply(assign_risk_tier)

tier_order = ['Low Risk (0-30%)', 'Medium Risk (30-60%)', 'High Risk (60-100%)']
summary = df.groupby('risk_tier').agg(
    num_accounts = ('account_id', 'count'),
    churned      = ('churn_flag', 'sum'),
    total_mrr    = ('avg_mrr', 'sum'),
).reindex(tier_order)
summary['churn_rate'] = (summary['churned'] / summary['num_accounts'] *
100).round(1)

for tier, row in summary.iterrows():
    print(f"{tier:<25} {int(row['num_accounts']):>4} accounts  ",
          f"{row['churn_rate']:>5.1f}% churn  ${row['total_mrr']:>10,.0f} MRR")
```

Output:

```
Low Risk (0-30%)           28 accounts    10.7% churn  $   111,740 MRR
Medium Risk (30-60%)      381 accounts    18.6% churn  $   864,230 MRR
High Risk (60-100%)        91 accounts    39.6% churn  $   156,118 MRR
```

Code Block A8. Risk segmentation of all 500 accounts. Observed churn rates rise monotonically, confirming directional validity of the model's probability scores.